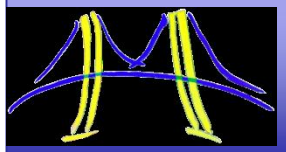
OpenMP

Part-2

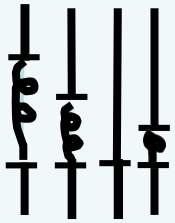
Contents on OpenMP

- OpenMP Execution Model
- Shared and private data
- Directives
- Barriers
- Sections
- Run-Time library functions
- Scheduling strategies
- Scalability study

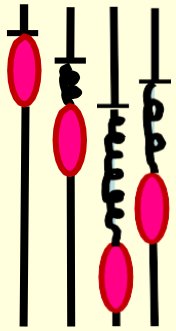


Synchronization:

- Synchronization: bringing one or more threads to a well defined and known point in their execution.
- The two most common forms of synchronization are:



Barrier: each thread wait at the barrier until all threads arrive.



Mutual exclusion: Define a block of code that only one thread at a time can execute.

Synchronization

- Synchronization is used to impose order constraints and to protect access to shared data

- High level synchronization:
  - **barrier**
  - **critical**
  - **atomic**
  - **reduction**

Synchronization: barrier/1

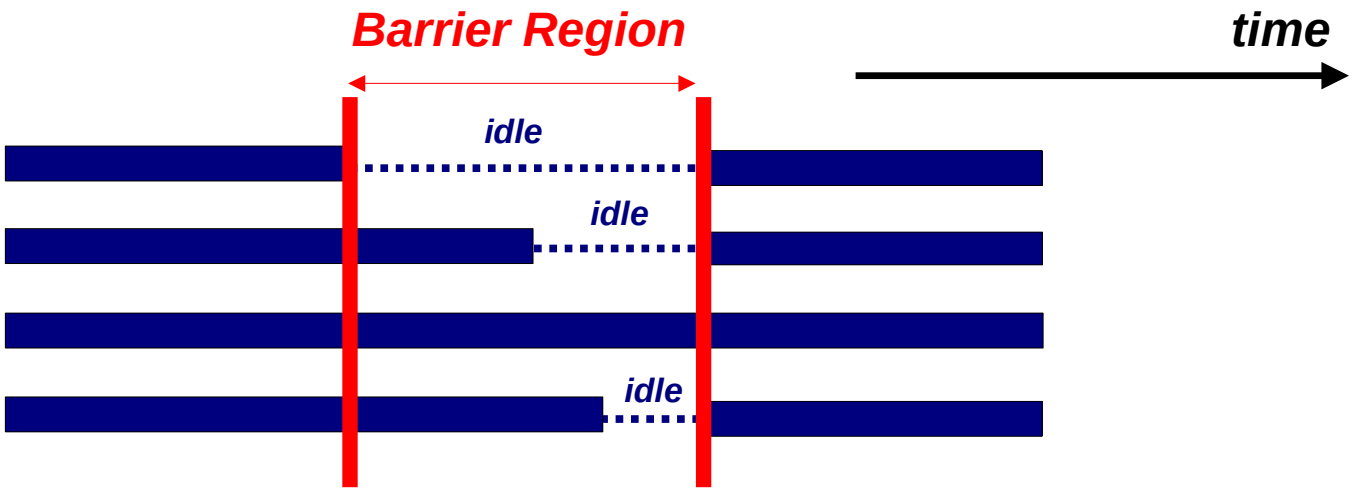
barrier clause: Each thread waits until all threads arrive.

```
#pragma omp parallel
{
        int id=omp_get_thread_num();
        A[id] = breakfast(id);
        #pragma omp barrier
        B[id] = PDC_Class(id, A);
}
```

// Friends taking breakfast

- All will come to class after food
- If anyone completes early, till all finishes they will wait.. (barrier)
- Then only they will participate into the PDC Class
- Same applicable to the Threads running in parallel.

Barrier/2

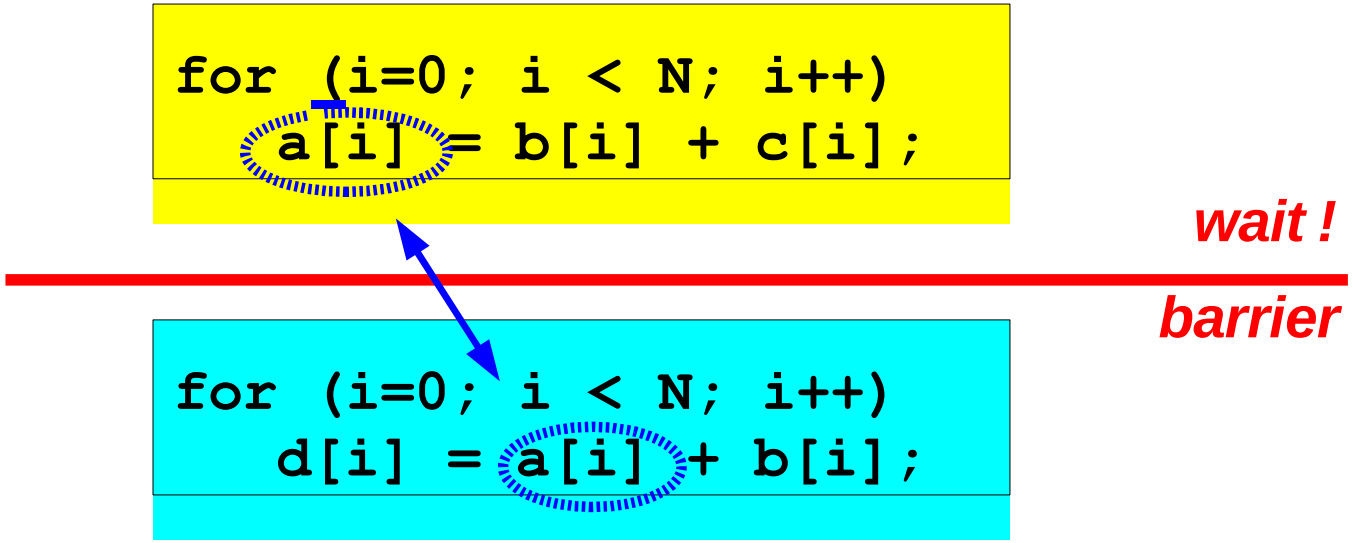


Each thread waits until all others have reached this point:

```
#pragma omp barrier
```

Barrier/3

We need to have updated all of a[ ] first, before using a[ ]



All threads wait at the barrier point and only continue when all threads have reached the barrier point

The nowait clause

- ***To minimize synchronization, some OpenMP directives/pragmas support the optional nowait clause***
- ***If present, threads will not synchronize/wait at the end of that particular construct***
- ***In C, it is one of the clauses on the pragma***

```
#pragma omp for nowait
{
        :
}
```

The nowait clause

- At the **end of a parallel region** the team of threads is **dissolved** and only the master thread continues.
- Therefore, there is an **implicit barrier** at the end of a parallel region.
- This **barrier behavior can be cancelled** with the **nowait** clause.

```
#pragma omp parallel
{
#pragma omp for nowait
  for (i=0; i<N; i++)
    a[i] = // some expression
#pragma omp for
  for (i=0; i<N; i++)
    b[i] = ..... a[i] .....
```

Here the **nowait** clause implies that threads can **start on the second loop** while other **threads are still working** on the first.

Synchronization: barrier

- **barrier**: Each thread waits until all threads arrive.

```
#pragma omp parallel shared (A, B, C) private(id)
{
        id=omp_get_thread_num();
        A[id] = big_calc1(id);
#pragma omp barrier
#pragma omp for
        for(i=0;i<N;i++){C[i]=big_calc3(i,A);}
#pragma omp for nowait
        for(i=0;i<N;i++){ B[i]=big_calc2(C,  i); }
        A[id] = big_calc4(id);
}
```

implicit barrier at the end of a for worksharing construct

implicit barrier at the end of a parallel region

no implicit barrier due to **nowait**

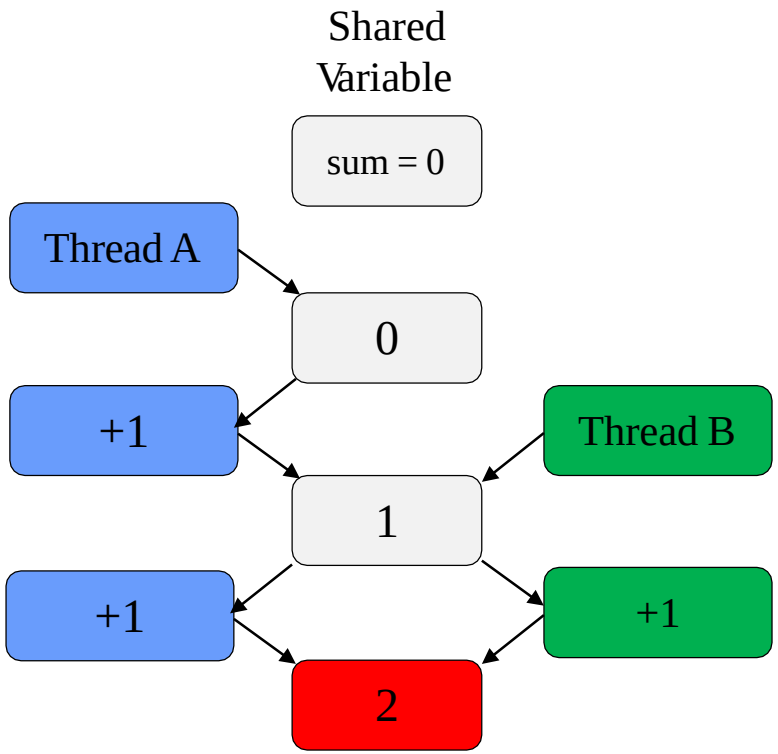
Critical Section

The code segment where race condition happened

Caution: Race Condition

- When multiple threads simultaneously read/write shared variable
- Multiple OMP solutions
  - Reduction
  - Atomic
  - Critical

```
#pragma omp parallel for private(i) shared(sum)
  for (i=0; i<N; i++) {
    sum += i;
  }
```



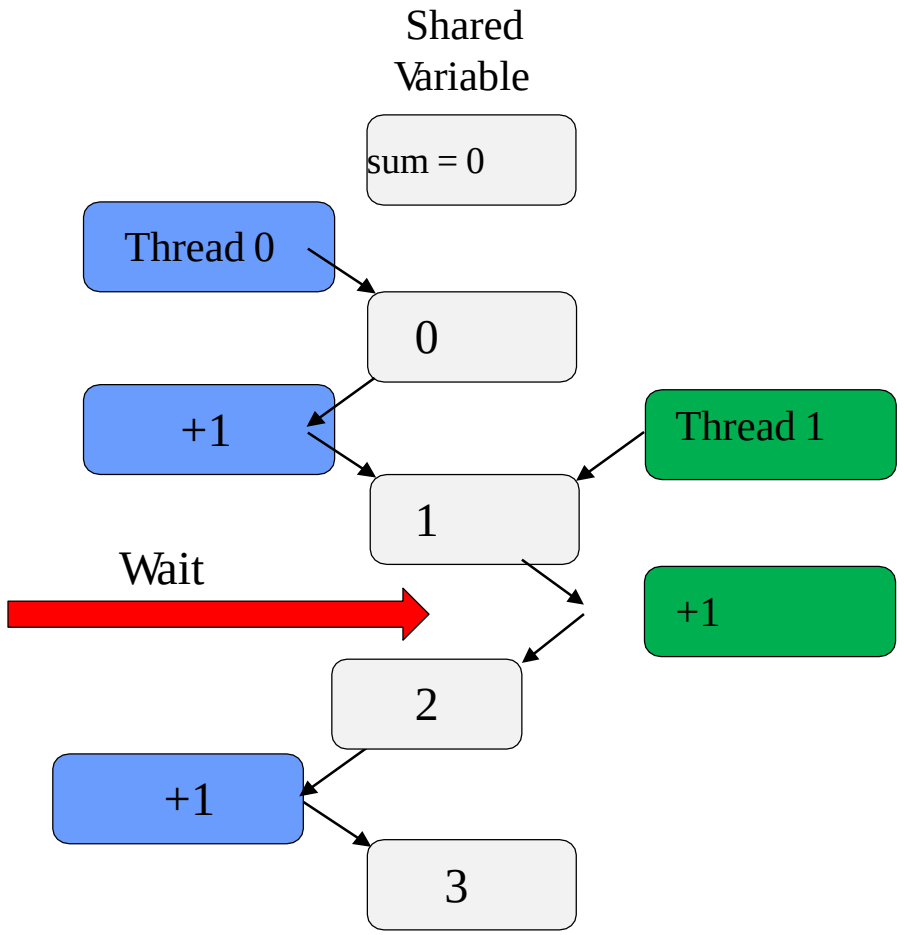
Should be 3!

Critical Section

- One solution: use **critical**
- Only one thread at a time can execute a critical section

```
#pragma omp critical
    {
      sum += i;
    }
```

- Downside?
  - SLOOOOWWW
  - Overhead & serialization

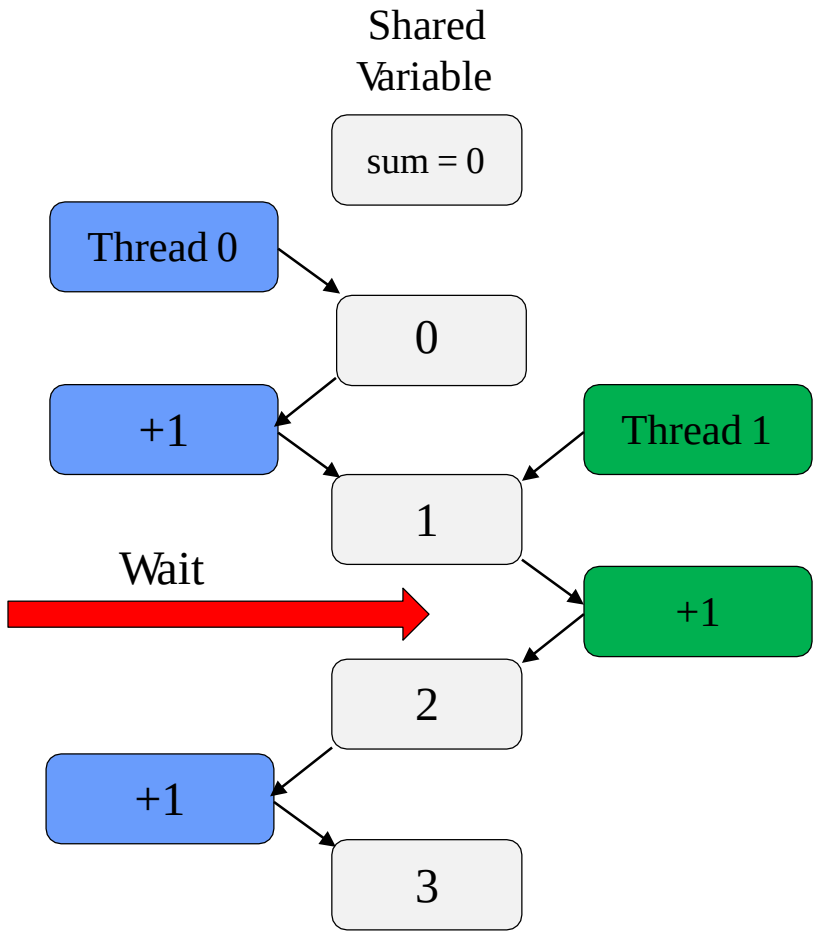


Atomic

- Atomic like “mini” critical
- Only one line
  - Certain limitations

```
#pragma omp atomic
    sum += i;
```

- Hardware controlled
  - Less overhead than critical



Reduction

```
#pragma omp reduction (operator:variable)
```

- Avoids race condition
- Reduction variable must be **shared**
- Makes variable private, then performs operator at end of loop
- Operator cannot be overloaded (c++)
  - One of: +, *, -, / (and &, ^, |, &&, ||)
  - OpenMP 3.1: added min and max for c/c++

Reduction Example

```
#include <omp.h>
#include <stdio.h>

int main() {

  int i;
  const int N = 1000;
  int sum = 0;

#pragma omp parallel for private(i) reduction(+: sum)
  for (i=0; i<N; i++) {
    sum += i;
  }

  printf("reduction sum=%d (expected %d)\n", sum, ((N-1)*N)/2);
```

```
[user@adroit3]$ gcc -fopenmp omp_race.c -o omp_race.out
[user@adroit3]$ export OMP_NUM_THREADS=4
[user@adroit3]$ ./omp_race.out
reduction sum=499500 (expected 499500)
```

Relative Performance

- For 4 threads:
  - Reduction is 100x faster than critical
  - Reduction is 10x faster than atomic
  - Reduction is faster than atomic with private sums (see example)

Verify this in Lab Sessions - to get to know execution time

```
start = omp_get_wtime();
{
    // code segment

}
end=omp_get_wtime();
exec_time = (end-start);
```

Runtime Library routines

- **Runtime environment routines:**
  - **Modify/Check the number of threads**
  - omp_set_num_threads(), omp_get_num_threads(), omp_get_thread_num(), omp_get_max_threads()
  - **Are we in an active parallel region?**
    - omp_in_parallel()
  - **Do you want the system to dynamically vary the number of threads from one parallel construct to another?**
    - omp_set_dynamic, omp_get_dynamic();
  - **How many processors in the system?**
    - omp_num_procs()

…plus a few less commonly used routines.

Runtime Library routines

- **To use a known, fixed number of threads in a program, (1)tell the system that you don't want dynamic adjustment of the number of threads, (2) set the number of threads, then (3) save the number you got.**

```
#include <omp.h>
void main()
{   int num_threads;
     omp_set_dynamic( 0 );
     omp_set_num_threads( omp_num_procs() );
#pragma omp parallel
   {     int id=omp_get_thread_num();
#pragma omp single
            num_threads = omp_get_num_threads();
         do_lots_of_stuff(id);
}
```

Disable dynamic adjustment of the number of threads.

Request as many threads as you have processors.

Protect this op since Memory stores are not atomic

Even in this case, the system may give you fewer threads than requested. If the precise # of threads matters, test for it and respond accordingly.

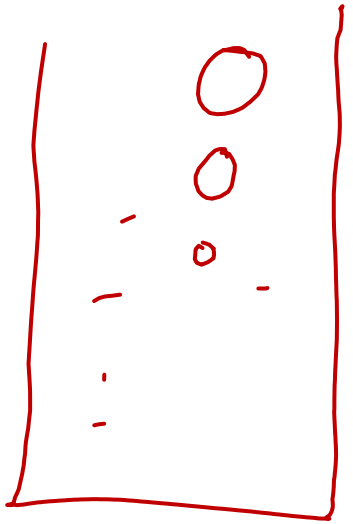
Environment Variables

- Set the default number of threads to use.
  - **OMP_NUM_THREADS ()**
- OpenMP added an environment variable to control the size of child threads' stack
  - **OMP_STACKSIZE()**
- Also added an environment variable to hint to runtime how to treat idle threads
  - **OMP_WAIT_POLICY**
    - **ACTIVE** **keep threads alive at barriers/locks**
    - **PASSIVE** **try to release processor at barriers/locks**

- Process binding is enabled if this variable is true … i.e. if true the runtime will not move threads around between processors.
  - **OMP_PROC_BIND true | false**

Odd - Even Transposition Sort

- Variant of Bubble Sort
- Compare and swap. – done



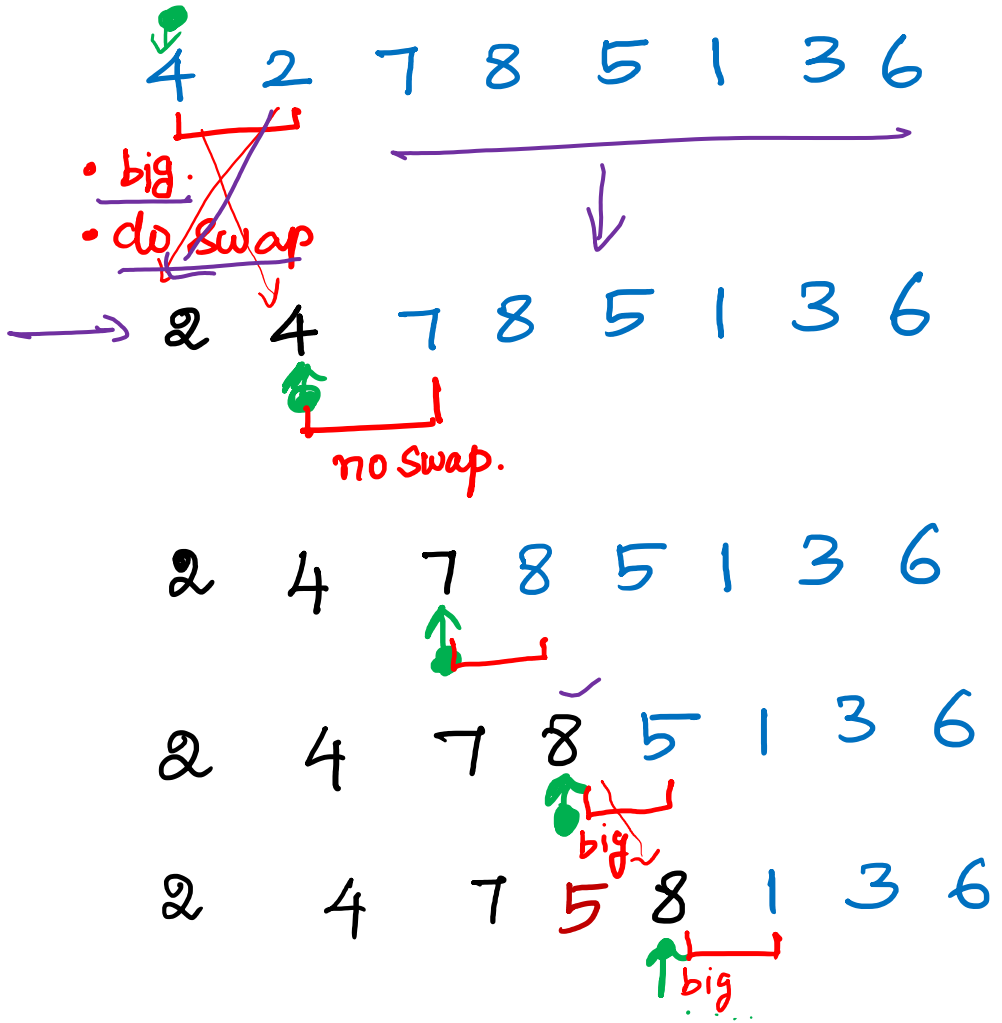
- Simplest sorting algorithm = known as Sorting by exchange (swap) & comparison-based algorithm.
- It works on the repeatedly swapping of adjacent elements until they are not in the intended order.
- Why bubble sort - the movement of array elements = the movement of air bubbles in the water. Bubbles in water rise up to the surface; similarly, the array elements => bubble sort highest number move to the end in each iteration.

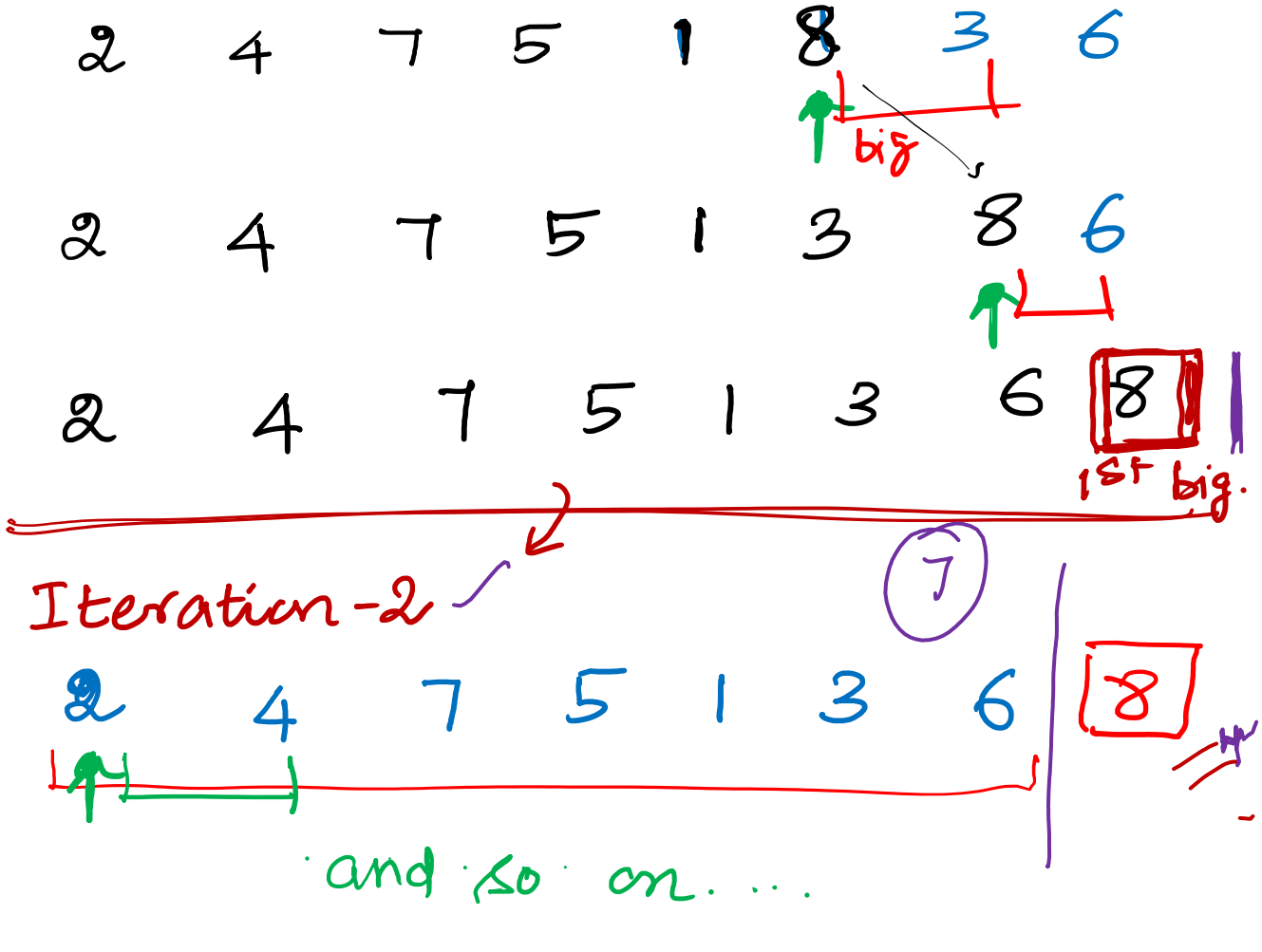
- Time complexity → $O(n^2)$ → n - no. of elements to be sorted.

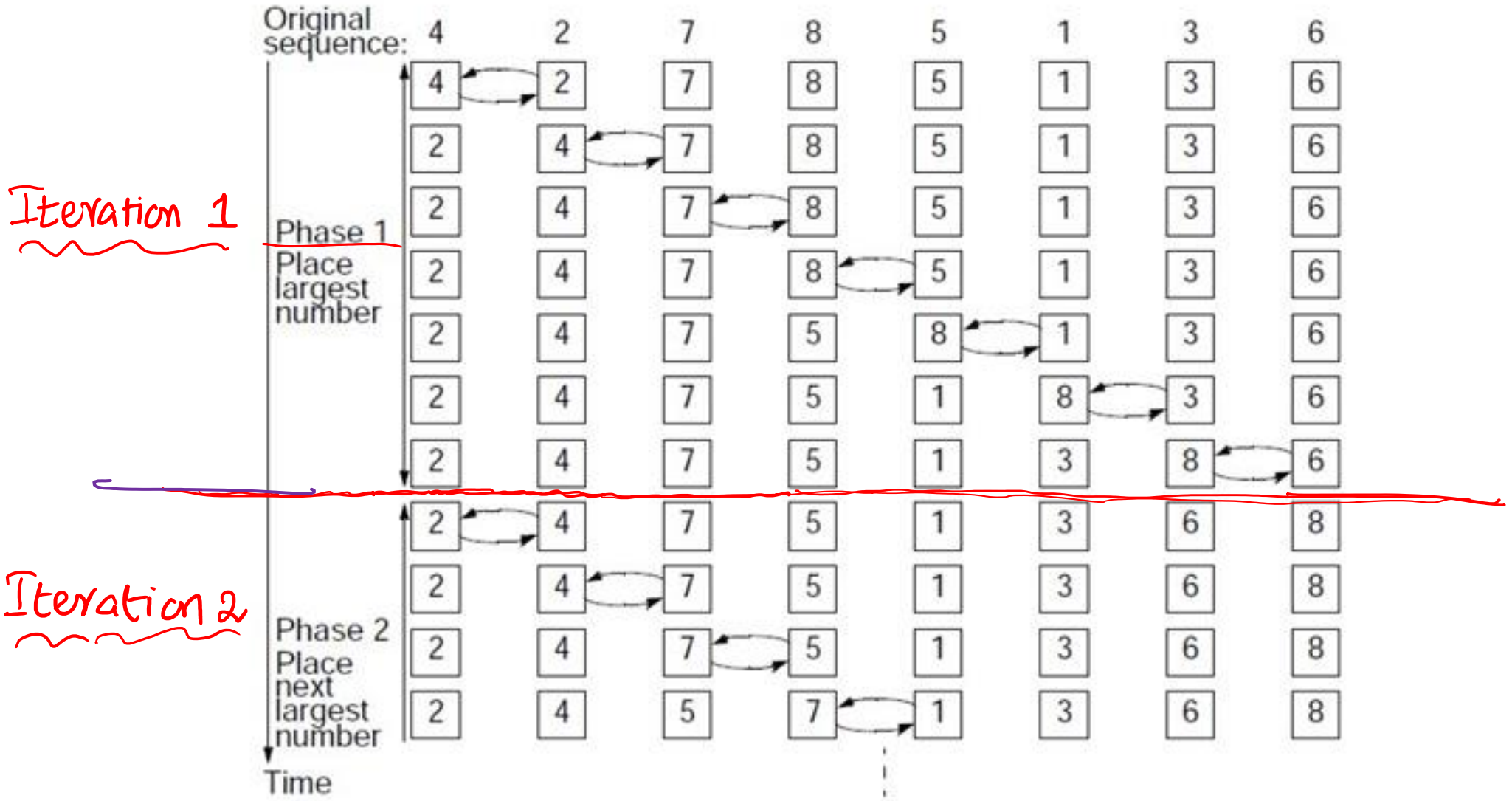
Bubble Sort (increasing)

Input: 4 2 7 8 5 1 3 6

Iteration 1: • → pointer to index.






Original sequence:
4 2 7 8 5 1 3 6
Iteration 1
Phase 1
Place largest number
Iteration 2
Phase 2
Place next largest number
Time

Sequential Bubble sort

```
for (list_length = n; list_length >= 2; list_length--) {
    for (i = 0; i < list_length; i++) {
        if (a[i] > a[i+1]) {
            tmp = a[i];
            a[i] = a[i+1];
            a[i+1] = tmp;
        }
    }
}
```

OET Sort

2 phases.

a

0	1	2	3	4	5	6	7	← Index.
4	2	7	8	5	1	3	6	

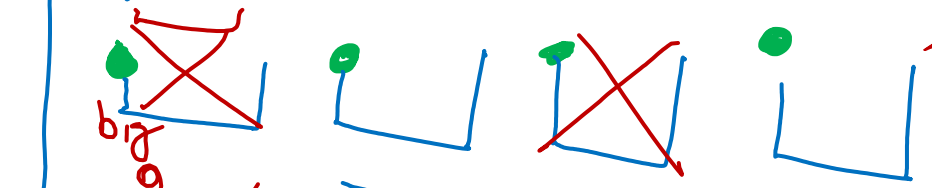
① Even phase – Compare and swap at 0, 2, 4, 6, ... (even indices) → to its right elt.

② Odd phase – " " 1, 3, 5, 7, ... (odd indices) → to its right elt.

Serial OET

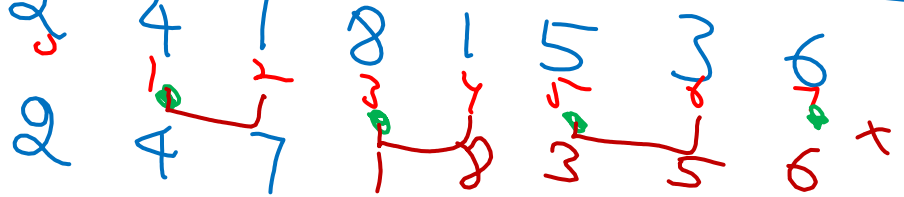
I_1 : Even Phase

0	1	2	3	4	5	6	7
4	2	7	8	5	1	3	6

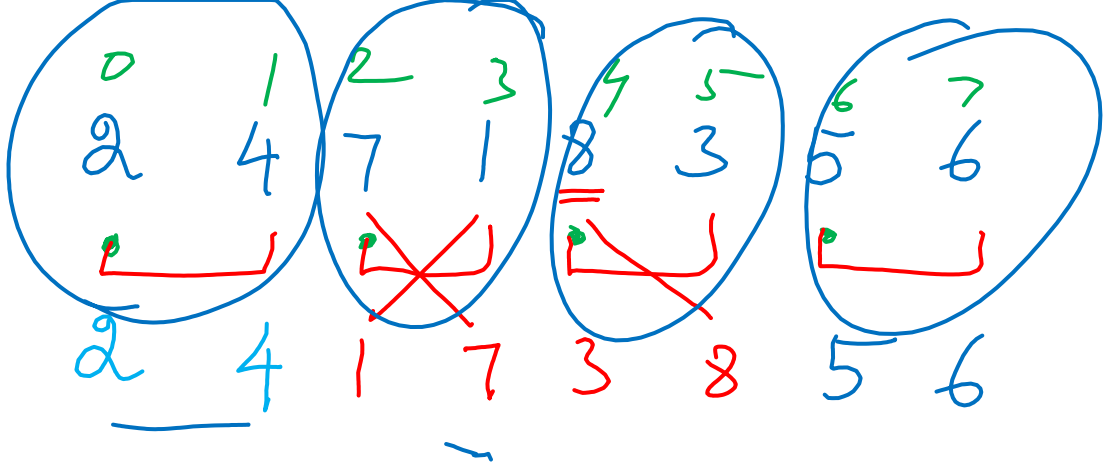


2	4	7	8	1	5	3	6

I_2 Odd phase



I_3 Even



I_4

I_8

Time complexity → $O(n^2)$ ↓ $O(n^2)$

$\|^{el}$

$=$ TC =

n

t

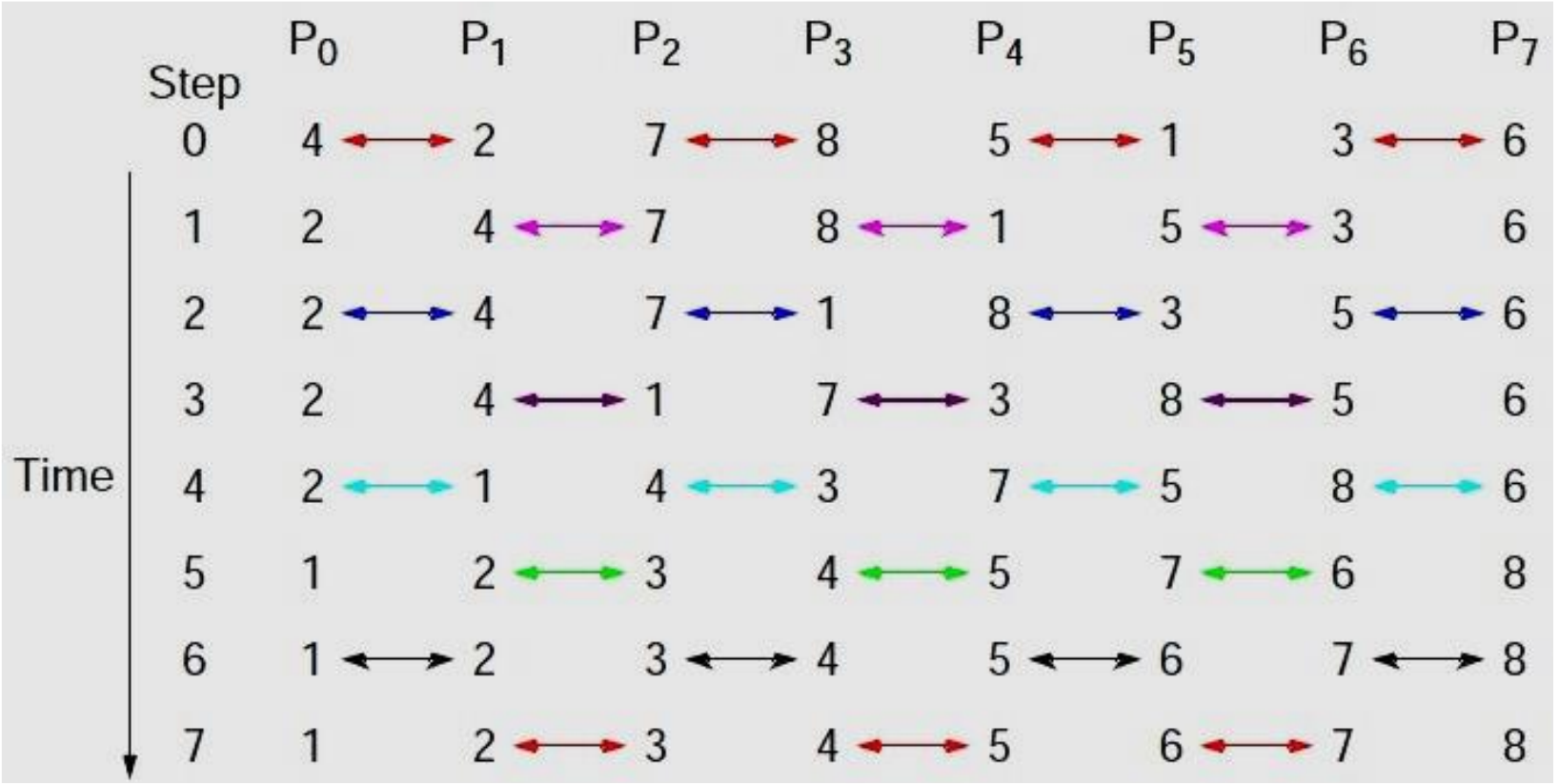
n & t

$$t = n$$

$$TC = ? \; O\left(\frac{n}{t} \times n\right)$$

$$O(n) = n$$

Odd-Even Transposition Sort - Sequential

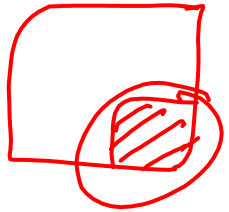


Parallel - First OpenMP Odd-Even Sort (P = n)

```
for (phase = 0; phase < n; phase++) {
   if (phase % 2 == 0)
      pragma omp parallel for num_threads(thread_count) \
         default(none) shared(a, n) private(i, tmp)
      for (i = 1; i < n; i += 2) {
         if (a[i-1] > a[i]) {
            tmp = a[i-1];
            a[i-1] = a[i];
            a[i] = tmp;
         }
      }
   else
      pragma omp parallel for num_threads(thread_count) \
         default(none) shared(a, n) private(i, tmp)
      for (i = 1; i < n-1; i += 2) {
         if (a[i] > a[i+1]) {
            tmp = a[i+1];
            a[i+1] = a[i];
            a[i] = tmp;
         }
      }
}
```

Monte Carlo Simulation — Estimate value of π

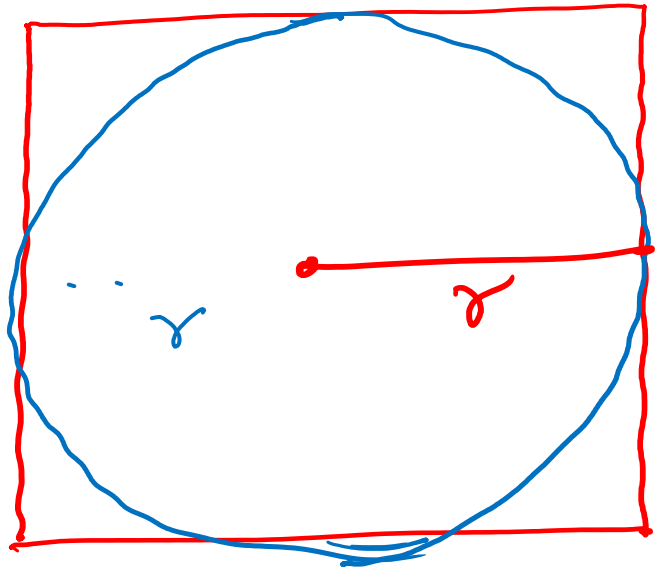
- MC method — algorithm — solves the problem through the use of Statistical sampling.



- Monaco city — famous for gampling. (Game of Chance).
  - ↑ name is derived from.
- 19th century — Dev. of atomic bomb — WW II.

How?

$$\pi = 4 \times \frac{\text{Area of Ou}}{\text{Area of } \square re}$$



Proof:

Area of circle of radius 'r'

(a, b)

$$(x-a)^2 + (y-b)^2 = r^2$$

$$\boxed{x^2 + y^2 = r^2}$$

(0,0)

Area of $O^{le} = \pi r^2$

Area of $\square^{re} = a^2 = (2r)^2 = 4r^2$

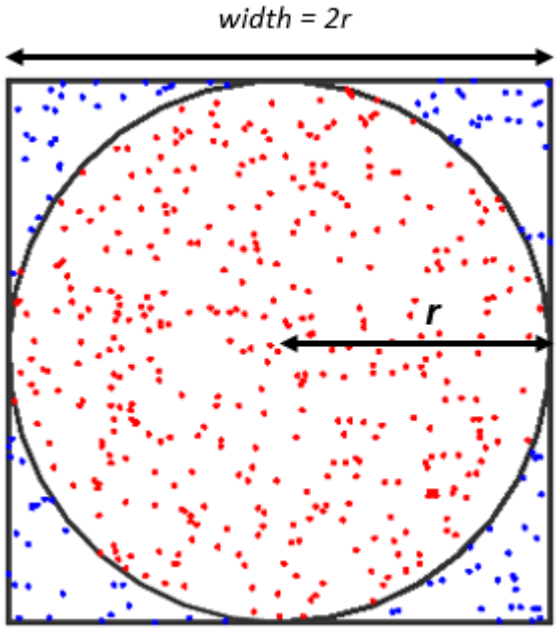
$$\frac{\text{Area of } \bigcirc^{le}}{\text{Area of } \square^{re}} = \frac{\pi r^2}{4r^2}$$

$$= \frac{\pi}{4}$$

$$4 \times \frac{\text{Area of } \bigcirc^{le}}{\text{Area of } \square^{re}} = \pi$$

$$\pi = 4 \times \frac{\text{Total No of Points in } \bigcirc^{le}}{\text{Total no of pts in } \square^{re}}$$

Estimating the value of Pi using Monte Carlo Simulation



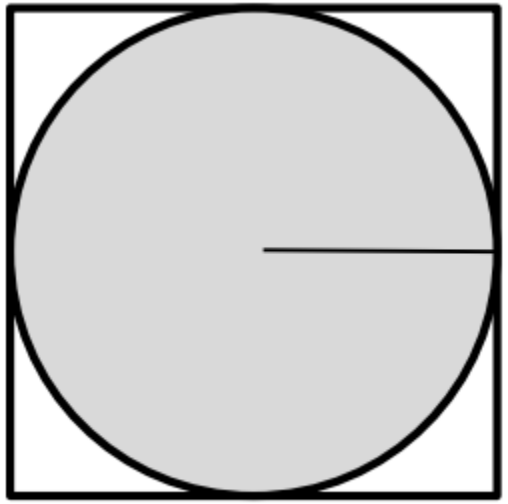
In the figure on the right the circle and the square have the following areas:

Area Circle: $A_C = \pi r^2$
Area Square: $A_S = (2r)^2 = 4r^2$

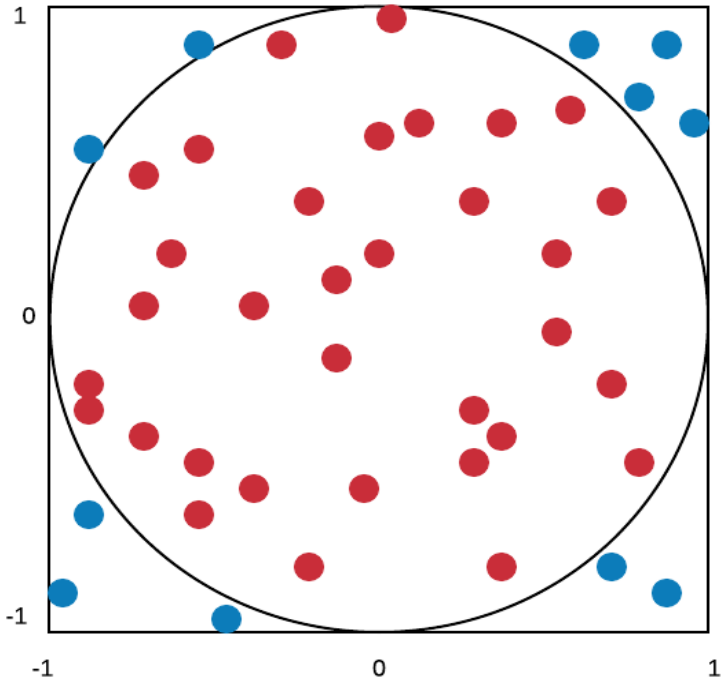
The ratio of the two areas is: $\frac{A_C}{A_S} = \frac{\pi r^2}{4r^2}$

And solving for π we get: $\pi = \frac{4A_C}{A_S}$

If we have an estimate for the ratio of the area of the circle to the area of the square we can solve for pi. The challenge becomes estimating this ratio.



Estimating the value of Pi using Monte Carlo Simulation



In the figure on the right the circle and the square have the following areas:

Area Circle: $A_C = \pi r^2$
Area Square: $A_S = (2r)^2 = 4r^2$

The ratio of the two areas is: $\frac{A_C}{A_S} = \frac{\pi r^2}{4r^2}$

And solving for π we get: $\pi = \frac{4A_C}{A_S}$

If we have an estimate for the ratio of the area of the circle to the area of the square we can solve for pi. The challenge becomes estimating this ratio.